SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Scenario-Based : 20%

QUESTIONS: Total Marks: 50 Annexure A

Code of Conduct:

I D.S.Tharuniya Priya (192511317) *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: *D.S.Tharuniya Priya*

Annexure A

Scenario-based

1. Secure Data Transmission using Block Cipher

A company is transmitting repeated financial data blocks over a network. Initially, they used ECB mode and noticed patterns in ciphertext.

A. Explain why this issue occurs.

B. Suggest a more secure block cipher mode and justify your choice based on the scenario.

A. Explain why this issue occurs.

Electronic Codebook (ECB) is a mode of operation that encrypts each plaintext block independently using the same secret key. A block cipher such as AES accepts a fixed-size block; if the message is longer than one block, it is divided into blocks and each block is encrypted separately. In ECB, the encryption of a block does not depend on the previous block, an initialization vector, or a nonce.

The central weakness is that identical plaintext blocks encrypted under the same key produce identical ciphertext blocks. In simplified notation, if P1 = P2, then ECB produces C1 = C2 because both blocks are processed using the same key and the same deterministic transformation. This repeated relationship remains visible even though the actual plaintext is not directly readable.

In the given company scenario, financial data may contain repeated structures such as transaction-status fields, currency codes, account-type labels, fixed headers, or padding. If the same block occurs repeatedly in the plaintext, the corresponding ciphertext block also repeats. An observer monitoring the network can therefore notice patterns, estimate where repeated fields occur, and infer relationships between records. The observer may not immediately learn an account number or transaction amount, but the visible structure leaks information about the message.

ECB also provides no integrity protection. An attacker may be able to copy, remove, reorder, or replace ciphertext blocks and cause changes in the decrypted message, depending on the application format. Because blocks are independent, such manipulation may not be detected automatically. Thus, ECB is unsuitable for protecting structured, repetitive financial records over an untrusted network.

The issue is not that the underlying block cipher is necessarily broken; the weakness is the mode of operation. A strong block cipher can still be used insecurely if it is applied in a mode that reveals repeated patterns or does not detect tampering.

B. Suggest a more secure block cipher mode and justify your choice based on the scenario.

A stronger choice is AES-GCM (Advanced Encryption Standard in Galois/Counter Mode). AES-GCM is an authenticated-encryption mode: it protects confidentiality by encrypting the data and provides integrity/authenticity through an authentication tag. This is particularly appropriate for financial information because the receiver needs both to keep transaction details secret and to detect unauthorized modification.

Unlike ECB, GCM uses counter-mode encryption with a nonce. When correctly implemented, the same plaintext does not simply map to the same ciphertext block in the way it does in ECB. The nonce makes each encryption operation distinct, so repeated financial fields do not expose the same obvious block patterns. The receiver also verifies the authentication tag before accepting the decrypted content. If an attacker changes ciphertext or protected associated data, verification should fail and the message should be rejected.

A practical transmission process is as follows: (1) the sender prepares the financial record; (2) the sender selects a nonce that is unique for the given key; (3) AES-GCM encrypts the record and produces an authentication tag; (4) the sender transmits the nonce, ciphertext, tag, and any required public metadata; (5) the receiver uses the same key and nonce to verify the tag and decrypt; and (6) the receiver accepts the record only if authentication succeeds. Sensitive fields should not be processed or acted upon before successful verification.

Nonce management is essential. A nonce must never be reused with the same AES-GCM key, because reuse can seriously undermine confidentiality and authentication. The system should use a reliable nonce-generation or counter strategy, protect keys, and define key rotation

procedures. If the organization cannot safely guarantee GCM nonce uniqueness, it should choose a well-reviewed authenticated-encryption construction whose requirements it can meet, rather than using GCM incorrectly.

Therefore, AES-GCM is recommended because it addresses both weaknesses relevant to the scenario: it avoids ECB-style pattern leakage and detects tampering. It is efficient in modern implementations and widely used for secure network communication. It should be deployed through a trusted cryptographic library with unique nonces, secure key storage, and rejection of failed authentication checks.

2. Choosing an Encryption Algorithm for Banking System

A banking application needs high security and moderate performance for encrypting customer transactions. The available options are DES, 3-DES, and Blowfish.

A. Identify the most suitable algorithm for this scenario.

B. Justify your choice based on security strength and efficiency.

A. Identify the most suitable algorithm for this scenario.

If the choice is strictly limited to DES, Triple DES (3-DES), and Blowfish, Blowfish is the most suitable of the three for the stated requirement of security with moderate performance. DES is not acceptable for modern protection because its effective key length is only 56 bits, making exhaustive key search practical for a capable attacker. Triple DES improves on DES by applying the DES operation three times, but it is computationally slower and has a 64-bit block size, which limits its suitability for high-volume modern banking traffic.

Blowfish was designed as a fast symmetric block cipher and supports variable-length keys from 32 bits up to 448 bits. With a properly selected strong key, it offers a much larger key space than DES and can perform efficiently in software. Therefore, among the listed options, Blowfish is the better fit for a historical or academic comparison where the decision must be made only from those three algorithms.

There is an important real-world qualification: Blowfish is not the preferred choice for a new banking system today. Its 64-bit block size can create collision risks when large volumes of

data are encrypted under the same key, and modern standards generally favor AES. If the bank is allowed to select an algorithm beyond the given list, AES with an authenticated mode such as AES-GCM would normally be recommended for new deployments. The answer of Blowfish applies only to the stated restricted options, not as a claim that it is the best current industry standard.

B. Justify your choice based on security strength and efficiency.

DES:

DES uses a 56-bit effective key. Although it was historically important, the key is too short to withstand modern brute-force attacks. An attacker can test possible keys until meaningful plaintext is recovered. DES should therefore not be used to protect confidential banking transactions, credentials, or customer information.

Triple DES:

3-DES applies DES encryption and decryption operations multiple times, commonly using two or three keys. This substantially increases resistance compared with single DES. However, it requires more processing than a single encryption operation and is consequently slower. It also retains DES's 64-bit block size; as the amount of encrypted data grows, the chance of block collisions becomes a concern. Triple DES is now considered a legacy choice and is being phased out in many modern applications.

Blowfish:

Blowfish permits a variable-length key, so an appropriately chosen key can resist exhaustive search far better than DES. It was designed for software efficiency and can provide reasonable performance for moderate workloads. These properties make it the strongest practical selection among the three options presented in the question. The key must be generated randomly and stored securely; using a weak password-derived key would undermine the benefit of a larger key space.

However, Blowfish's 64-bit block size is an important limitation for a bank processing a large number of transactions. With enough blocks encrypted under one key, repeated ciphertext blocks and birthday-bound collision effects become increasingly likely. The organization would

need to consider data volume, key lifetime, and mode of operation, and should not use ECB. A modern authenticated mode and current standard cipher are preferable where available.

Decision:

Choose Blowfish only when the bank must select from DES, 3-DES, and Blowfish. It provides a better balance of key strength and software efficiency than DES and 3-DES in this restricted comparison. For an actual present-day banking deployment, select a currently approved algorithm such as AES-GCM, follow applicable standards, and use professional cryptographic libraries rather than designing the cryptographic implementation independently.

3. Key Exchange in an Unsecured Network

Two users want to establish a secure communication channel over a public network using Diffie-Hellman key exchange. However, the network is vulnerable to attacks.

A. Explain how the key exchange works in this scenario.

B. Identify a possible security threat and suggest a solution to mitigate it.

A. Explain how the key exchange works in this scenario.

Diffie–Hellman (DH) is a key-agreement protocol that allows two parties to derive the same shared secret while communicating over a public network. The secret itself is not transmitted. The shared secret can then be passed through a key-derivation function to produce one or more symmetric session keys for encrypting and authenticating the actual communication.

Step 1 — Public parameters: Alice and Bob agree on public mathematical parameters, traditionally a large prime number p and a generator g. These values do not need to be secret. In real systems, standardized groups or an approved elliptic-curve Diffie–Hellman variant should be used rather than inventing parameters.

Step 2 — Alice creates her private value: Alice selects a fresh, unpredictable private number a and computes A = g^a mod p. She keeps a secret and sends the public value A to Bob.

Step 3 — Bob creates his private value: Bob independently selects a fresh, unpredictable private number b and computes B = g^b mod p. He keeps b secret and sends the public value B to Alice. An eavesdropper may observe p, g, A, and B, but should not be able to feasibly calculate the shared secret when secure parameters and a sufficiently strong group are used.

Step 4 — Alice derives the shared secret: Alice computes K = B^a mod p. Bob independently computes K = A^b mod p. Since B = g^b mod p and A = g^a mod p, both calculations yield g^(ab) mod p, so Alice and Bob obtain the same shared value without sending K across the network.

Step 5 — Derive session keys: The raw DH result should not normally be used directly as an encryption key. Both parties apply a suitable key-derivation function, incorporating context and, where appropriate, transcript information, to derive symmetric keys. Those keys can then be used with an authenticated-encryption algorithm to protect messages. Fresh ephemeral DH values can also provide forward secrecy when the protocol is designed and implemented correctly.

The mathematical exchange establishes a shared secret, but basic Diffie–Hellman by itself does not prove who is on the other end of the connection. This distinction is crucial in the scenario because the network is vulnerable to active attacks.

B. Identify a possible security threat and suggest a solution to mitigate it.

The main threat is a man-in-the-middle (MITM) attack. Basic Diffie–Hellman does not authenticate the exchanged public values. An attacker positioned between Alice and Bob can intercept Alice's value A and send the attacker's own value to Bob; the attacker can also intercept Bob's value B and send a different attacker-controlled value to Alice. Alice may then establish a secret with the attacker, while Bob establishes a separate secret with the attacker. The attacker can decrypt, read, alter, and re-encrypt messages between the parties without either user realizing that the connection has been intercepted.

The principal mitigation is authentication. Alice and Bob should authenticate the DH exchange using digital signatures linked to trusted certificates, or use a well-designed authenticated key-exchange protocol such as a correctly configured TLS connection. Each participant must

validate the peer's certificate, identity, signature, and protocol parameters. Merely encrypting the DH values is not enough if the parties cannot verify who supplied them.

Additional protections include using standardized, cryptographically strong parameters; generating fresh private values with a secure random-number generator; using a secure key-derivation function; and protecting the resulting session keys. The protocol should include transcript binding or key confirmation where appropriate, so that the keys are tied to the intended session and peer identities. Implementations should rely on maintained cryptographic libraries and should reject invalid public values.

Therefore, Diffie–Hellman is useful because it allows a shared secret to be created without transmitting that secret. However, it must be combined with authentication and correct protocol design to resist an active attacker on an unsecured network. Authenticated Diffie–Hellman provides both the key-agreement benefit and confidence that the key is shared with the intended party.

4. Public Key Cryptography for Email Security

An organization plans to implement RSA for secure email communication.

A. Describe how RSA can be used for encryption and decryption in this scenario.

B. Explain how key distribution and management play a role in maintaining security.

A. Describe how RSA can be used for encryption and decryption in this scenario.

RSA is an asymmetric cryptographic algorithm. Each user has a related public key and private key. The public key can be distributed to other users, while the private key must remain confidential and under the owner's control. RSA can support key transport/encryption and digital signatures, but these are distinct operations and should use appropriate standardized schemes.

Step 1 — Key generation: Bob's email client or organizational key-management system generates an RSA key pair. The public key is made available to senders, while the private key is stored securely and accessible only to Bob or an authorized cryptographic service. In a real

deployment, key sizes, padding schemes, and certificate requirements must follow current security guidance.

Step 2 — Obtain the recipient's verified public key: If Alice wants to send Bob a confidential email, she obtains Bob's public key and verifies that it genuinely belongs to Bob. This verification is essential: encrypting to an attacker's substituted public key would not protect the email from that attacker.

Step 3 — Protect the message using hybrid encryption: In practical secure email, Alice normally does not encrypt the entire email directly with RSA. Instead, her email software generates a random symmetric content-encryption key and encrypts the email body and attachments using an authenticated symmetric algorithm. It then uses an appropriate RSA encryption scheme, such as RSA-OAEP where supported by the protocol, to protect the small symmetric key for Bob. This hybrid method is more efficient because symmetric encryption is much faster for large messages.

Step 4 — Send the protected email: Alice sends the encrypted message, the RSA-protected content key, and the metadata required by the secure-email format. The protected key and ciphertext may travel over a public network because an eavesdropper who lacks Bob's private key should not be able to recover the content key or read the message.

Step 5 — Decryption by Bob: Bob's email software uses his private RSA key and the appropriate decryption scheme to recover the symmetric content key. It then uses that key to decrypt and authenticate the email content. If authentication fails, the software should treat the message as invalid rather than displaying or processing untrusted plaintext as if it were authentic.

Step 6 — Verify the sender when required: Confidentiality alone does not prove who sent a message. To support authenticity and integrity, Alice can digitally sign the message or its digest using her private signing key with a suitable RSA signature scheme, such as RSA-PSS where supported. Bob verifies the signature using Alice's trusted public key. A valid signature provides evidence that the corresponding private signing key was used and that the signed content has not changed; it does not, by itself, guarantee that the human sender's account or device was uncompromised.

B. Explain how key distribution and management play a role in maintaining security.

Public-key distribution must establish the link between a key and its owner. Although public keys are not secret, an attacker may substitute a different public key and trick a sender into encrypting information for the attacker. Organizations therefore need a trustworthy way to publish, verify, and update users' public keys.

A Public Key Infrastructure (PKI) can support this through digital certificates. A certificate binds an identity to a public key and is signed by a Certificate Authority (CA) trusted by the recipient. Before relying on a certificate, the email client should validate the certificate chain, validity period, intended usage, identity information, and revocation status as required by the system. In some environments, a securely managed organizational directory or an independently verified fingerprint may also be used.

Private-key protection is equally important. Private keys should be generated with approved tools and strong randomness, stored in encrypted storage or hardware-backed devices where appropriate, and protected by access controls and secure authentication. Access should be limited to authorized users and services. Private signing keys and decryption keys may need different access, backup, and recovery policies because they serve different purposes.

Key lifecycle management includes generation, secure distribution of public keys, renewal, rotation, revocation, backup, recovery, and eventual destruction. If an employee leaves the organization, a device is lost, or a private key is suspected to be compromised, the organization must have a process to revoke or replace the affected key and inform correspondents. Revocation information must be available to systems that rely on the key; otherwise, a compromised certificate may continue to be trusted.

The organization should also select secure RSA parameters and standardized padding, prevent reuse of keys for inappropriate purposes, maintain software updates, and document recovery procedures. Decryption-key backup may be required for business continuity, but it must be tightly controlled because a backup is another target for attackers. Digital signatures should be verified separately from encryption, and users should be trained to respond correctly to certificate warnings.

In conclusion, RSA contributes to email confidentiality and can support digital signatures, but the complete system is only as secure as its key distribution and management. Trusted identity

verification, strong private-key protection, appropriate algorithms, revocation, and disciplined lifecycle procedures are necessary to prevent impersonation, disclosure, and misuse.

5. Cryptography for IoT Devices

A smart home system uses low-power IoT devices that need secure communication. The designer must choose between RSA and Elliptic Curve Cryptography (ECC).

A. Analyze the constraints of IoT devices.

B. Recommend a suitable cryptographic approach and justify your answer.

A. Analyze the constraints of IoT devices.

Smart-home IoT devices such as door sensors, smart locks, cameras, thermostats, and lighting controllers often have much more limited resources than desktop computers or servers. Cryptography must therefore provide strong security without exhausting processing capacity, memory, battery power, or network bandwidth. The consequences of failure can also be significant because devices may control physical access or reveal household activity.

1. Limited processing power: Many IoT devices use small microcontrollers with modest clock speeds and limited hardware acceleration. Public-key operations, especially with large RSA keys, can require more computation than is practical for frequent handshakes or very small devices. Slow operations can increase response time and delay security checks.

2. Limited RAM and storage: Devices may have small amounts of memory for code, protocol state, certificates, keys, and temporary data. Larger keys and certificates increase storage and transmission requirements. Cryptographic libraries must be selected carefully so that implementation overhead does not exceed the device's resources.

3. Battery and energy constraints: Some sensors are battery-powered and expected to operate for months or years without maintenance. Repeated computation and radio transmission consume energy. A design that performs expensive operations too frequently may shorten battery life, so session reuse, efficient key establishment, and sensible rekeying policies are important.

4. Restricted bandwidth and intermittent connectivity: IoT devices may communicate over low-rate wireless links or networks with packet loss and limited message sizes. Large public keys, certificates, and protocol handshakes create extra traffic and latency. The protocol should minimize unnecessary data while still preserving authentication and security.

5. Physical exposure and long deployment life: Devices may be installed in accessible locations and remain in service for years. An attacker might attempt physical access, extract secrets, or exploit outdated firmware. Devices therefore need secure boot where available, protected key storage, authenticated firmware updates, vulnerability patching, and a plan for replacing or revoking compromised credentials.

6. Heterogeneous hardware and safety requirements: A smart home may combine devices from different manufacturers with different capabilities and update schedules. The system must use interoperable, well-tested protocols and avoid weak custom cryptography. Security decisions should consider the impact of compromise, particularly for door locks, alarms, and devices that handle sensitive data.

These constraints mean that the best design is not simply the algorithm with the smallest computation cost. It must balance security strength, memory use, communication overhead, energy consumption, maintainability, and the consequences of an attack.

B. Recommend a suitable cryptographic approach and justify your answer.

Elliptic Curve Cryptography (ECC) is generally the more suitable choice between RSA and ECC for public-key operations on low-power IoT devices. ECC can provide security comparable to RSA with substantially smaller public keys for commonly used security levels. Smaller keys reduce storage needs and the number of bytes transmitted during key exchange, which is helpful for limited-memory devices and low-bandwidth networks.

For example, an elliptic-curve key around 256 bits is commonly used to provide a security level broadly comparable to RSA with a much larger key, such as 3072 bits. Exact equivalence depends on the scheme and security assumptions, but the general benefit is that ECC uses compact keys. ECC-based key agreement, such as a standardized elliptic-curve Diffie–Hellman scheme, can establish a shared secret with lower communication overhead than a comparable RSA-based key-transport approach.

A recommended design is to use ECC for authenticated key establishment and then use symmetric authenticated encryption for routine data. The device and hub first authenticate each other using certificates, pinned public keys, or another secure credential system. They perform an approved ECC-based key agreement to derive a shared secret, use a key-derivation function to produce session keys, and protect subsequent messages using an authenticated-encryption algorithm supported by the device, such as AES-GCM when its nonce requirements can be met. Where hardware and protocol requirements differ, an approved alternative may be selected by the system designer.

This hybrid approach is efficient because public-key cryptography is mainly used during setup or rekeying, while fast symmetric cryptography protects the repeated sensor messages. Authentication prevents unauthorized devices from joining the network, encryption protects confidentiality, and integrity checks help detect modified commands or sensor readings. For critical commands, the receiver should reject messages that fail authentication and should use anti-replay protections, such as sequence numbers or protocol-defined counters.

ECC is not automatically secure merely because its keys are small. The implementation must use standardized curves and schemes, strong random-number generation, validated cryptographic libraries, secure key storage, authenticated protocols, and a supported firmware-update mechanism. Private keys should not be hard-coded identically into every device. If a device is captured or compromised, credentials should be revocable and the system should be able to re-enrol or replace it securely.

Decision: choose ECC over RSA for the stated low-power smart-home environment because its smaller keys and lower communication overhead generally fit constrained devices better. Combine ECC-based authenticated key establishment with symmetric authenticated encryption for ongoing traffic. This provides a practical balance of security, performance, bandwidth use, and energy consumption while maintaining a clear path for key updates and device management.

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand